```
OpenDaylight-user@root>feature:list | grep networkoptimizer
odl-networkoptimizer-api         | 1.0.0-SNAPSHOT       | x
| odl-networkoptimizer-1.0.0-SNAPSHOT       | OpenDaylight ::
networkoptimizer :: api
odl-networkoptimizer             | 1.0.0-SNAPSHOT       | x
| odl-networkoptimizer-1.0.0-SNAPSHOT       | OpenDaylight ::
networkoptimizer
odl-networkoptimizer-rest        | 1.0.0-SNAPSHOT       | x
| odl-networkoptimizer-1.0.0-SNAPSHOT       | OpenDaylight ::
networkoptimizer :: REST
odl-networkoptimizer-ui          | 1.0.0-SNAPSHOT       | x
| odl-networkoptimizer-1.0.0-SNAPSHOT       | OpenDaylight ::
networkoptimizer :: UI
OpenDaylight-user@root>
```

Shut down the Karaf container, as follows:

```
OpenDaylight-user@root>shutdown -f
```

YANG definition for RPC

As shown below, YANG defines an RPC named service-account with inputs and outputs.

```
module networkoptimizer {
    yang-version 1;
    namespace "urn:OpenDaylight:params:xml:ns:yang:networkop
timizer";
    prefix "networkoptimizer";
    revision "2015-01-05" {
        description "Initial revision of networkoptimizer
model";
    }
    rpc service-account{
        input {
              leaf name {
                    type string;
              }
              leaf description {
                    type string;
              }
        }
        output {
             leaf service_name {
                   type string;
             }
        }
    }
}
```

On re-building the application with this YANG, it would generate Java Objects and interfaces corresponding to the attributes defined in YANG. Please refer to the following files, which are auto generated under the API sub-directories.

```
├── NetworkoptimizerService.java
├── ServiceAccountInputBuilder.java
├── ServiceAccountInput.java
├── ServiceAccountOutputBuilder.java
├── ServiceAccountOutput.java
├── $YangModelBindingProvider.java
└── $YangModuleInfoImpl.java
```

The Eclipse set-up

In Eclipse, import the network optimiser directory generated as the Maven project. This would create different projects in Eclipse for different top folders, in which you can add the 'api' project as dependency to 'impl'.

Implementing RPC

Provide an implementation for the service interface generated by *yangtools* from the YANG definition of *serviceaccount*. You can create the file *NetworkOptimizerImpl.java*, which provides that implementation under *impl/src/main/java/com/cloudenablers/networkoptimizer/impl/* and paste the following contents in it:

```
package com.cloudenablers.networkoptimizer.impl;

import java.util.concurrent.Future;
import org.OpenDaylight.yang.gen.v1.urn.OpenDaylight.
params.xml.ns.yang.networkoptimizer.rev150105.
NetworkoptimizerService;
import org.OpenDaylight.yang.gen.v1.urn.OpenDaylight.params.
xml.ns.yang.networkoptimizer.rev150105.ServiceAccountInput;
import org.OpenDaylight.yang.gen.v1.urn.OpenDaylight.params.
xml.ns.yang.networkoptimizer.rev150105.ServiceAccountOutput;
import org.OpenDaylight.yang.gen.v1.urn.OpenDaylight.
params.xml.ns.yang.networkoptimizer.rev150105.
ServiceAccountOutputBuilder;

import org.OpenDaylight.yangtools.yang.common.RpcResult;
import org.OpenDaylight.yangtools.yang.common.
RpcResultBuilder;

public class NetworkoptimizerImpl implements
NetworkoptimizerService {

     @Override
     public Future<RpcResult<ServiceAccountOutput>> serviceAc
count(ServiceAccountInput input) {
       ServiceAccountOutputBuilder serviceAccBuilder = new
ServiceAccountOutputBuilder();
       serviceAccBuilder.setServiceName("RPC request
successful for Service Name: " + input.getName());
       return RpcResultBuilder.success(serviceAccBuilder.
build()).buildFuture();
     }
}
```